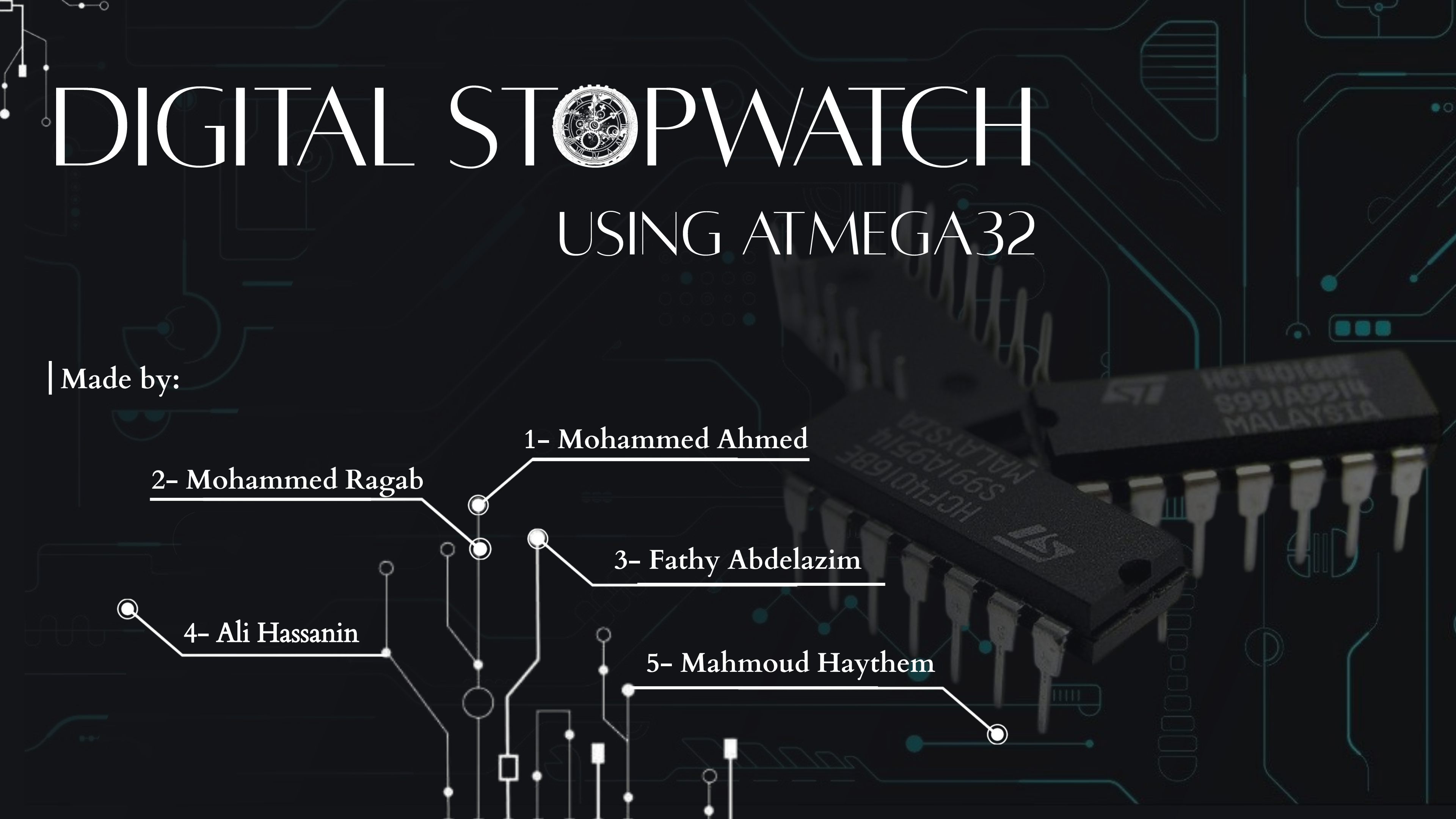




DIGITAL STOPWATCH

USING ATMEGA32

| Made by:

1- Mohammed Ahmed

2- Mohammed Ragab

3- Fathy Abdelazim

4- Ali Hassanin

5- Mahmoud Haythem

Goal of the Project

- Countdown timer that can count from hours, minutes down to seconds
- The user enters the time using a keypad, and the LCD shows what they type as a guide
- The six seven-segment displays show the time counting down in real time
- When the time reaches zero, a buzzer sounds to tell us the time is up

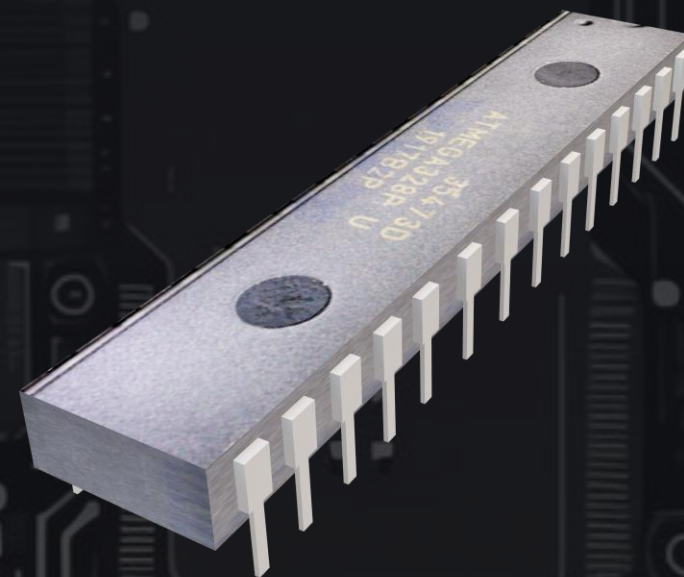
Components



6 Pin 7Segment
Displays



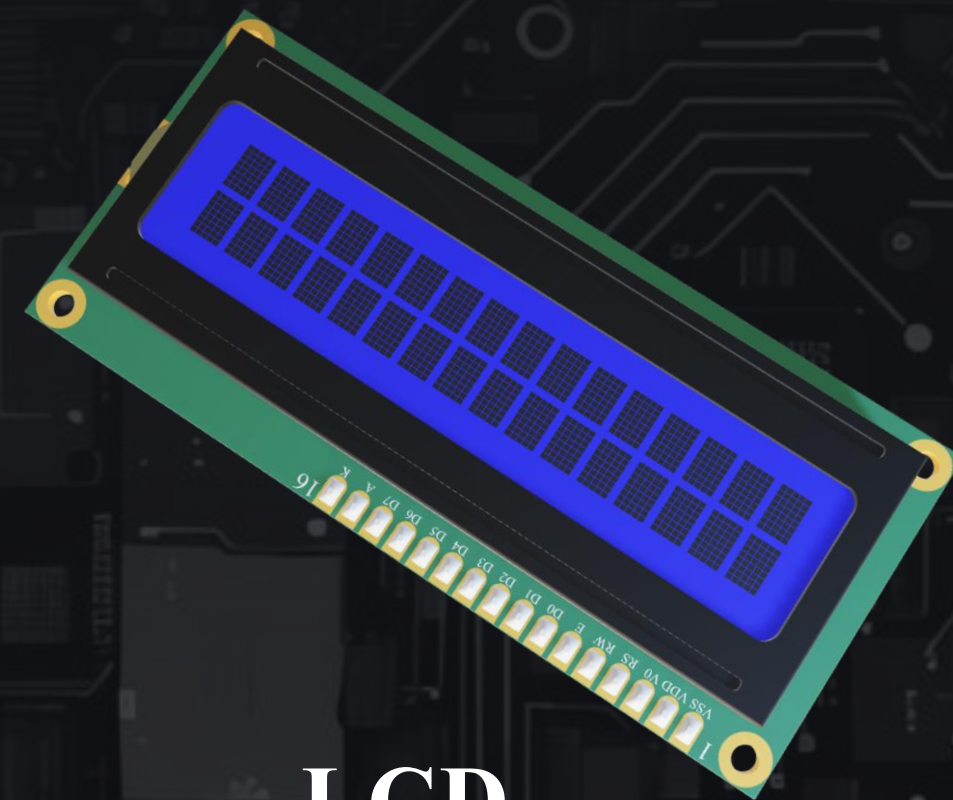
4x4 Keypad



ATmega32



Buzzer



LCD

Challenges We Faced

Challenge 1: Six Digits, Not Enough Pins

SOLUTION

Multiplexing

We tied all 6 digits' segment pins together on one port (PORTA) and gave each digit its own on/off pin on a second port (PORTC). We light only ONE digit at a time, but cycle through all 6 so fast that the human eye sees all of them glowing at once.

In Code

```
*7Seg.c
122
123 void H7SEG_voidRefresh(Seg_t A_xSeg)
124 {
125     u8 L_u8CommValue = 0x00 ;
126     u8 L_u8Number    = 0x00 ;
127
128     if (A_xSeg.Type == COMM_CATHODE)
129         L_u8CommValue = 0xFF;
130     else
131         L_u8CommValue = 0x00;
132     MDIO_voidSetPortValue(A_xSeg.Comm_Port, L_u8CommValue);
133
134
135     L_u8Number = GP_u8Digits[G_u8Active];
136     if (L_u8Number <= 9)
137         MDIO_voidSetPortValue(A_xSeg.Data_Port, G_u8SegNumbers[L_u8Number]);
138     else
139         MDIO_voidSetPortValue(A_xSeg.Data_Port, 0x00);
140
141
142     if (A_xSeg.Type == COMM_CATHODE)
143         CLR_BIT(L_u8CommValue, G_u8Active);
144     else
145         SET_BIT(L_u8CommValue, G_u8Active);
146     MDIO_voidSetPortValue(A_xSeg.Comm_Port, L_u8CommValue);
147
148
149     _delay_ms(2);
150     G_u8Active++;
151     if (G_u8Active > H7SEG_DIG5)
152         G_u8Active = H7SEG_DIG0;
153 }
154
```

Challenges We Faced

SOLUTION

We used APP_MODE_ALARM. When the time reaches 00:00:00, the program enters the alarm state and calls HBuzzer_voidBuzzerTog() to sound the alert.

Challenge 2:

The buzzer should only sound at the exact moment the countdown hits 00:00:00, not earlier, and not while the timer is still running.

In Code

```
*main.c
266 static void App_voidTick(void)
267 {
268     if (G_u8SecFlag == 1)
269     {
270         G_u8SecFlag = 0;
271
272         if (G_u8Mode == APP_MODE_COUNTDOWN)
273         {
274             if (G_u8SS > 0) G_u8SS--;
275             else
276             {
277                 G_u8SS = APP_SS_MAX;
278                 if (G_u8MM > 0) G_u8MM--;
279                 else
280                 {
281                     G_u8MM = APP_MM_MAX;
282                     if (G_u8HH > 0) G_u8HH--;
283                 }
284             }
285
286             if (G_u8HH == 0 && G_u8MM == 0 && G_u8SS == 0)
287             {
288                 G_u8Mode = APP_MODE_ALARM;
289                 HBuzzer_voidBuzzerOff(DIO_PORTB, DIO_PIN3);
290                 HLCD_voidClrScreen();
291                 HLCD_voidMoveCursor(LCD_ROW0, LCD_COL4);
292                 HLCD_voidDisplayStr((u8*)"TIME UP!");
293             }
294         }
295     }
296     else if (G_u8Mode == APP_MODE_ALARM)
297     {
298         HBuzzer_voidBuzzerTog(DIO_PORTB, DIO_PIN3);
299     }
300 }
301 }
```

Challenges We Faced

SOLUTION

While scanning the keypad matrix, we read the pin twice with a 20 ms delay in between. If the key is still pressed after the delay, we accept it as a real press. Then we also wait for the key to be released before reading again, so holding the key doesn't register it over and over.

Challenge 3 :

A single button press might be registered multiple times due to button bouncing.

In Code

```
*KP.c x
32 u8 HKP_u8GetPressedValue (u8 A_u8PortName)
33 {
34     u8 L_u8Row ;
35     u8 L_u8Col ;
36     for (L_u8Col = 0 ; L_u8Col < 4 ; L_u8Col ++ )
37     {
38         MDIO_voidSetPinValue(A_u8PortName, L_u8Col + 4, DIO_LOW) ;
39         for (L_u8Row = 0 ; L_u8Row < 4 ; L_u8Row ++ )
40         {
41             if (MDIO_u8GetPinValue(A_u8PortName, L_u8Row) == KP_PRESSED)
42             {
43                 _delay_ms(20) ;
44                 if (MDIO_u8GetPinValue(A_u8PortName, L_u8Row) == KP_PRESSED)
45                 {
46                     while (1)
47                     {
48                         while (MDIO_u8GetPinValue(A_u8PortName, L_u8Row) == KP_PRESSED) ;
49                         _delay_ms(20) ;
50                         if (MDIO_u8GetPinValue(A_u8PortName, L_u8Row) != KP_PRESSED)
51                         {
52                             break ;
53                         }
54                     }
55                     MDIO_voidSetPinValue(A_u8PortName, L_u8Col + 4, DIO_HIGH) ;
56                     return G_u8KpData [L_u8Col][L_u8Row] ;
57                 }
58             }
59             else
60             {
61                 MDIO_voidSetPinValue(A_u8PortName, L_u8Col + 4, DIO_HIGH) ;
62                 return KP_NOT_PRESSED ;
63             }
64         }
65         MDIO_voidSetPinValue(A_u8PortName, L_u8Col + 4, DIO_HIGH) ;
66     }
67     return KP_NOT_PRESSED ;
68 }
69
```

Challenges We Faced

Challenge 4 :

Counting down:

If we used `_delay_ms(1000)` to wait one second, the CPU would be stuck doing nothing for that whole second. While it's paused, the 7-segment displays wouldn't refresh (they'd go dark/flicker) and the LCD would freeze.

SOLUTION

We use Timer1 in CTC (Compare) mode to count a real second in the hardware. When exactly 1 second passes, the CPU is interrupted and the ISR runs a callback that only sets a flag (`G_u8SecFlag = 1`). The main loop is free to keep running the displays; it just checks the flag and only then does the decrement.

In Code

Timer1 setup — interrupt every 1 second

```
*Timer1.c ✕
03
04 void MTimer1_voidSetDesiredTime (u32 A_u32TimeInMicro , void (*fptr)(void))
05 {
06     u32 L_u32Value ;
07
08     L_u32Value = (u32)( ( (u64)A_u32TimeInMicro * 8000000UL ) / ( (u32)G_u32Timer1Prescaler * 1000000UL ) ) ;
09
10     OCR1A = L_u32Value ;
11     Timer1_ptr = fptr ;
12
13     SET_BIT(SREG , 7) ;
14     SET_BIT(TIFR , 4) ;
15     SET_BIT(TIMSK , 4) ;
16 }
17
18
```

Callback + ISR

```
*Timer1.c ✕
81
82 void __vector_7 (void)__attribute__((signal)) ;
83 void __vector_7 (void)
84 {
85     if (Timer1_ptr != 0)
86     {
87         Timer1_ptr () ;
88     }
89 }
90
```

Challenges We Faced

Challenge 5 :

The user needs to see the digits being entered during timer setup.

SOLUTION

We used the LCD to display SET TIMER and the user input, and implemented `App_voidEnterDigit()` to update the displayed number.

In Code

```
*main.c *
55 static void App_voidEnterDigit (u8 A_u8Key)
56 {
57     G_u8Digits [G_u8SetupPos] = (u8)(A_u8Key - '0') ;
58     HLCD_voidMoveCursor(LCD_ROW1 , G_u8SlotCol [G_u8SetupPos]) ;
59     HLCD_voidWriteData(A_u8Key) ;
60     G_u8SetupPos ++ ;
61 }
62
63 static void App_voidCheckPair (void)
64 {
65     u8 L_u8Last  = (u8)(G_u8SetupPos - 1) ;
66     u8 L_u8Pair  = (u8)(L_u8Last / 2) ;
67     u8 L_u8Start = (u8)(L_u8Pair * 2) ;
68     u8 L_u8Max   = 0 ;
69     u8 L_u8Value = 0 ;
70
71     if (L_u8Pair == 0)
72     {
73         L_u8Max = APP_HH_MAX ;
74     }
75     else if (L_u8Pair == 1)
76     {
77         L_u8Max = APP_MM_MAX ;
78     }
79     else
80     {
81         L_u8Max = APP_SS_MAX ;
82     }
83
84     L_u8Value = (u8)(G_u8Digits [L_u8Start] * 10 + G_u8Digits [L_u8Start + 1]) ;
85
86     if (L_u8Value > L_u8Max)
87     {
88         G_u8Digits [L_u8Start] = 0 ;
89         G_u8Digits [L_u8Start + 1] = 0 ;
90         G_u8SetupPos = L_u8Start ;
91         HLCD_voidMoveCursor(LCD_ROW1 , G_u8SlotCol [L_u8Start]) ;
92         HLCD_voidWriteData('_') ;
93         HLCD_voidMoveCursor(LCD_ROW1 , G_u8SlotCol [L_u8Start + 1]) ;
94         HLCD_voidWriteData('_') ;
95     }
96 }
```

Pinout

Port	Pin(s)	Function	Direction
PORTA	PA0 – PA7	7-Segment display: segment data (a–g, dp), common-cathode	Output
PORTB	PB0	LCD: RS	Output
PORTB	PB1	LCD: R/W	Output
PORTB	PB2	LCD: E	Output
PORTB	PB3	Buzzer drive	Output
PORTB	PB4 – PB7	LCD: Data lines D4–D7	Output
PORTC	PC0 – PC5	7-Segment display: (common)	Output
PORTD	PD0 – PD3	Keypad: row inputs	Input
PORTD	PD4 – PD7	Keypad: column outputs	Output

Final product

